Collegium Witelona
Wydział nauk technicznych i ekonomicznych

Metody Sztucznej Inteligencji



CS Loadout Helper - System Ekspertowy

Jacek Kowalczyk
nr albumu 44921
grupa s3PAM1(1)

Spis treści:

1. **Wstęp** - str. 3
2. **Opis projektu** - str. 4
   - 2.1. Cel aplikacji
   - 2.2. Wymagania projektu
   - 2.3. Wykorzystane technologie
3. **Działanie systemu ekspertowego** - str. 6
   - 3.1. Pamięć robocza
   - 3.2. Baza wiedzy
   - 3.3. Maszyna wnioskowania
   - 3.4. Diagram działania
4. **Implementacja kodu** - str. 9
   - 4.1. Struktura aplikacji
   - 4.2. Skrypt app.js
   - 4.3. Skrypt interferenceEngine.js
   - 4.4. Skrypt knowledgeBase.js
5. **Interfejs użytkownika** - str. 12
   - 5.1. System design i inspiracje
   - 5.2. Graficzny interfejs użytkownika
6. **Podsumowanie**- str. 14
   - 6.1. Wnioski
   - 6.2. Możliwości rozwoju
7. **Bibliografia** - str. 15
   - 7.1. Tabela diagramów
   - 7.2. Tabela rysunków

1. Wstęp

Współczesne gry e-sportowe, w tym Counter-Strike 2, charakteryzują się wysoką dynamiką rozgrywki oraz złożonym systemem ekonomiczno-taktycznym. Kluczowym elementem decydującym o powodzeniu drużyny jest odpowiednie zarządzanie funduszami oraz dobór uzbrojenia adekwatnego do umiejętności użytkownika i drużynowego planu na daną rundę.

Projekt CS2 Loadout Helper to regułowy system ekspertowy (Rule-Based Expert System), którego zadaniem jest symulacja procesu decyzyjnego gracza. Na podstawie zestawu wprowadzonych faktów taktycznych system dokonuje automatycznej dedukcji i wskazuje optymalną broń główną wraz z uzasadnieniem wyboru.

2. Opis projektu

2.1. Cel aplikacji

Głównym celem projektu jest stworzenie inteligentnego systemu doradczego wspomagającego dobór uzbrojenia w grze Counter-Strike 2 z wykorzystaniem klasycznych metod sztucznej inteligencji. Do szczegółowych celów należą:

- Formalne wymodelowanie wiedzy dziedzinowej dotyczącej balansu broni, metagry oraz ekonomii CS2 w postaci bazy faktów i reguł produkcyjnych.
- Implementacja silnika wnioskowania w przód (Forward Chaining), operującego na logice "if… then…" z mechanizmem rozwiązywania konfliktów decyzyjnych (Conflict Resolution).
- Zagwarantowanie racjonalności ekonomicznej – eliminacja błędów poznawczych (np. unikanie rekomendacji taniej broni startowej przy pełnym budżecie).

2.2. Wymagania projektu

W ramach realizacji projektu sformułowano następujące wymagania funkcjonalne i niefunkcjonalne:

- Architektura regułowa i separacja warstw: Całkowite odseparowanie Bazy Wiedzy (knowledgeBase.js), Maszyny Wnioskowania (inferenceEngine.js) oraz Kontrolera Interfejsu (app.js).
- Pełne pokrycie dziedziny: Baza wiedzy musi obejmować wszystkie 34 dostępne w grze bronie (pistolety, pistolety maszynowe, karabiny szturmowe, karabiny snajperskie, strzelby i ciężkie karabiny maszynowe).
- Dwuetapowy proces diagnostyczny: Wybór frakcji (Terroryści / Antyterroryści) determinujący dostępną pulę uzbrojenia i motyw wizualny oraz ankieta taktyczna zbierająca 5 kluczowych kryteriów (stan budżetu, rolę taktyczną, planowany dystans starcia, pożądane cechy konstrukcyjne oraz preferowany styl strzelania).
- Jednoznaczność i czytelność wyniku: Prezentacja dokładnie jednej, najbardziej optymalnej broni wraz z jej specyfikacją techniczną (obrażenia, szybkostrzelność, penetracja pancerza, itp.), wektorową ikoną HUD oraz renderem modelu 3D w formie obrazu rastrowego.
- Brak zależności zewnętrznych: Aplikacja musi działać w pełni po stronie klienta (Client-Side SPA), bez konieczności uruchamiania serwerów backendowych, instalacji środowiska Node.js czy pobierania pakietów przez menedżery zależności.

2.3. Wykorzystane technologie

Projekt został zrealizowany w oparciu o technologie natywne środowiska przeglądarkowego (Vanilla Web Stack):

- HTML5: Semantyczna struktura strony, formularze diagnostyczne oparte na polach wyboru typu radio oraz struktura paneli modali.
- CSS3: Dedykowany, autorski system stylów inspirowany taktycznym interfejsem HUD gry Counter-Strike 2. Wykorzystano zaawansowane mechanizmy CSS Variables (dynamiczne przełączanie motywów T/CT), CSS Grid (prezentacja parametrów technicznych broni) oraz Flexbox.
- JavaScript (ES6+): Implementacja algorytmów sztucznej inteligencji (maszyna wnioskowania w przód, pamięć robocza faktów, scoring wagowy reguł), dynamiczna manipulacja drzewem DOM i reaktywne przejścia między panelami bez odświeżania strony (SPA).

3. Działanie systemu ekspertowego

System doradczy został zrealizowany w oparciu o klasyczny paradygmat systemu opartego na wiedzy (Rule-Based Expert System). Jego architektura realizuje ścisłą separację pomiędzy zgromadzoną wiedzą dziedzinową a procesami obliczeniowymi i dedukcyjnymi.

2.1. Pamięć robocza

Pamięć robocza przechowuje stan bieżącego problemu decyzyjnego w postaci zestawu 6 atrybutów kategorialnych (faktów pierwotnych):

- side (Frakcja): określa przynależność drużynową i ogranicza przestrzeń poszukiwań do broni dostępnych dla danej strony lub neutralnych.
- round_economy (Budżet rundy): kluczowy fakt ekonomiczny definiujący dopuszczalne półki cenowe.
- tactical_role (Rola taktyczna): specyfika zadania gracza w rundzie.
- engagement_distance (Dystans starć): geometria przewidywanych pojedynków ogniowych.
- weapon_feature (Cechy konstrukcyjne): preferowane cechy wyposażenia (np. tłumik, powiększony magazynek, celownik optyczny).
- playstyle (Styl walki): mechanika strzelania preferowana przez gracza.

2.2. Baza wiedzy

Baza wiedzy składa się z dwóch powiązanych komponentów:

- Katalogu obiektów (34 bronie): Pełny zestaw parametrów numerycznych i opisowych każdej broni w grze (cena zakupu, nagroda za zabójstwo, obrażenia bazowe, mnożnik w głowę, szybkostrzelność, penetracja pancerza, pojemność magazynka, kontrola odrzutu).
- Bazy reguł produkcyjnych (34 reguły): Każda z broni ma przypisaną regułę, od której zależy, czy zostanie ona wzięta pod uwagę i czy może pojawić się jako wynik końcowy po uzupełnieniu ankiety.

3.3. Maszyna wnioskowania

Maszyna wnioskowania realizuje algorytm wnioskowania w przód (Forward Chaining) – proces prowadzony jest w kierunku od danych wejściowych (faktów) do konkluzji (celu).

W pierwszej fazie silnik iteruje po zbiorze reguł produkcyjnych i eliminuje reguły niespełniające warunków brzegowych:

- Zgodność frakcyjna: Odrzucane są reguły dedykowane przeciwnej drużynie.
- Twarde ograniczenie budżetowe: Reguła musi odpowiadać bieżącemu poziomowi ekonomii gracza.
- Wprowadzono elastyczność dla graczy z nadmiarem funduszy: przy budżecie powyżej $5000 system akceptuje również reguły z poziomu full (np. AWP, AK-47, M4A1-S), zapobiegając sztucznemu ograniczaniu wyboru wyłącznie do najdroższych autosnajperek.
- Wszystkie pozostałe kombinacje budżetowe są natychmiast odrzucane, co całkowicie wyklucza nielogiczne rekomendacje.

Dla każdej reguły, która przeszła wstępną filtrację, silnik oblicza stopień dopasowania (matchScore):

- +10 pkt za dokładną zgodność budżetu.
- +4 pkt za dopuszczenie broni z poziomu full przy funduszach rich.
- +3 pkt za każdą kolejną przesłankę zgodną z faktem podanym przez gracza.
- -5 pkt nakładana jednorazowo, jeżeli reguła definiuje przesłankę, w której użytkownik wskazał w ankiecie inną wartość. Zapewnia to, że reguła o najwyższej specyficzności zawsze wygrywa z regułą o cechach częściowo sprzecznych.

W sytuacji, gdy przesłanki więcej niż jednej reguły zostaną spełnione (konflikt w zbiorze konfliktowym), maszyna wnioskowania stosuje strategię Sortowania według stopnia dopasowania (Priority / Match Specificity) - wówczas wszyscy kandydaci porządkowani są malejąco według obliczonego wyniku. Do egzekucji wybierana jest reguła z pozycji pierwszej.

W przypadku podania skrajnie nietypowej lub niespójnej kombinacji odpowiedzi, dla której żadna reguła produkcyjna nie zakwalifikuje się do zbioru kandydatów, uruchamia się awaryjny Fallback Mechanism. Zwraca on bezpieczną, uniwersalną broń domyślną przypisaną na stałe do danej frakcji i poziomu zamożności (np. AK-47 dla T i M4A1-S dla CT przy full, Galil/FAMAS przy semi, MAC-10/MP9 przy force, Glock/USP przy eco). Gwarantuje to stuprocentową odporność systemu na zawieszenie i brak odpowiedzi.

3.4 Diagram działania

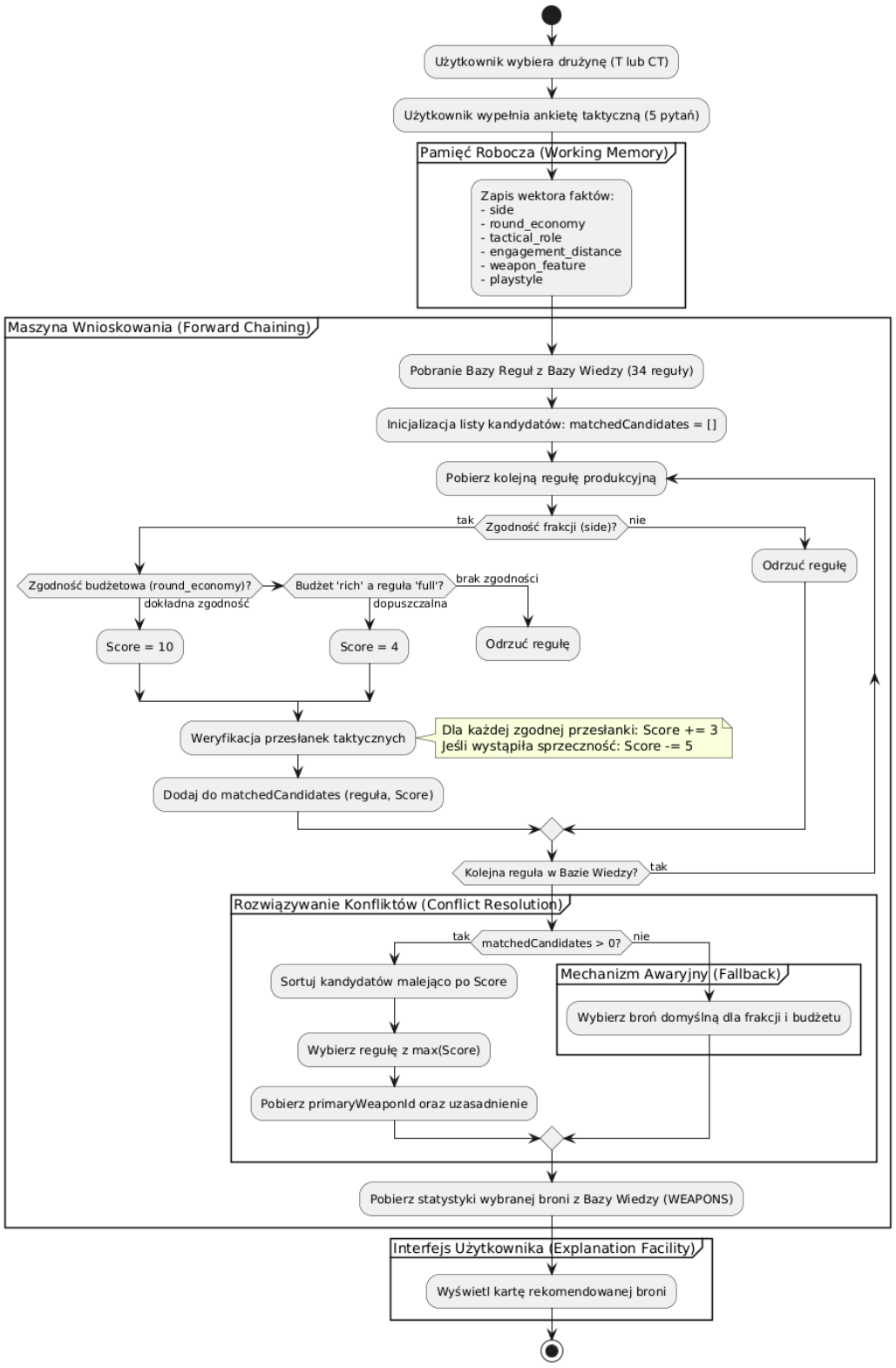


Diagram nr 1: “Diagram działania systemu ekspertowego”

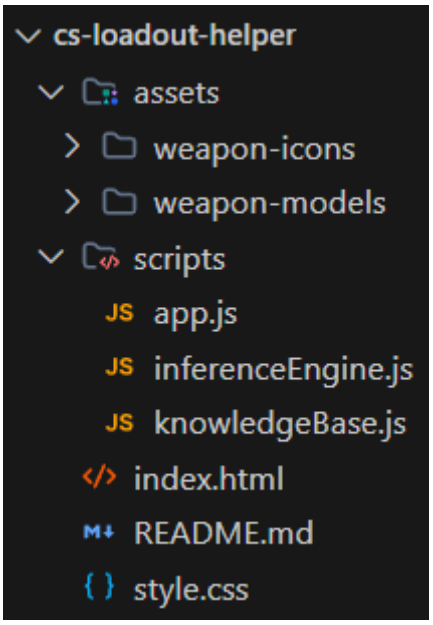
4. Implementacja kodu

Implementacja systemu opiera się na modularnej strukturze w języku JavaScript (standard ES6+), działającej w pełni synchronicznie po stronie klienta. Logika aplikacji została zaprojektowana z zachowaniem ścisłej separacji danych od algorytmów decyzyjnych oraz warstwy prezentacji, co ułatwia testowanie i rozbudowę reguł bez ingerencji w interfejs użytkownika.

Sercem implementacji jest algorytm wnioskowania w przód, który iteruje po strukturach reguł produkcyjnych, filtruje dopuszczalnych kandydatów w oparciu o stan pamięci roboczej i wyznacza optymalne rozwiązanie przy użyciu funkcji punktacji dopasowania. Całość współpracuje z reaktywnym kontrolerem interfejsu (SPA), który przechwytuje zdarzenia formularza, dynamicznie manipuluje drzewem DOM i renderuje wyniki wraz z parametrami technicznymi broni.

4.1 Struktura aplikacji

Pliki javascript to skrypty odpowiadające za algorytm systemu ekspertowego oraz kontroler interfejsu. To one są najważniejsze w kontekście tego projektu i ich fragmenty zamieszczam poniżej. W katalogu assets znajdują się wszystkie ikony oraz rendery każdej broni dostępnej w grze. index.html oraz style.css to oczywiście szablon i stylizacja strony internetowej.



Rysunek nr 1: “Struktura aplikacji”

4.2. Plik app.js

scripts > JS app.js > ...

```
1   (function () {
100   // Powrót do wyboru drużyny
101   function returnToTeamSelection() {
102     if (appState.autoTransitionTimer) {
103       clearTimeout(appState.autoTransitionTimer);
104       appState.autoTransitionTimer = null;
105     }
106
107     // Reset motywu do domyślnego
108     document.body.className = "";
109     appState.facts.side = null;
110     if (dom.hiddenSideInput) dom.hiddenSideInput.value = "";
111
112     // Wyczyszczenie pytań ankiety
113     if (dom.form) {
114       dom.form.reset();
115       const allRadios = dom.form.querySelectorAll('input[type="radio"]');
116       allRadios.forEach(radio => {
117         radio.checked = false;
118       });
119     }
120     TACTICAL_QUESTION_KEYS.forEach(key => {
121       appState.facts[key] = null;
122     });
123
124     // Pokazanie panelu wyboru drużyny
125     if (dom.surveyPanel) dom.surveyPanel.style.display = "none";
126     if (dom.resultPanel) dom.resultPanel.style.display = "none";
127     if (dom.teamSelectPanel) dom.teamSelectPanel.style.display = "block";
128
129     window.scrollTo({ top: 0, behavior: "smooth" });
130   }
```

Rysunek nr 2: “Fragment kodu pliku app.js”

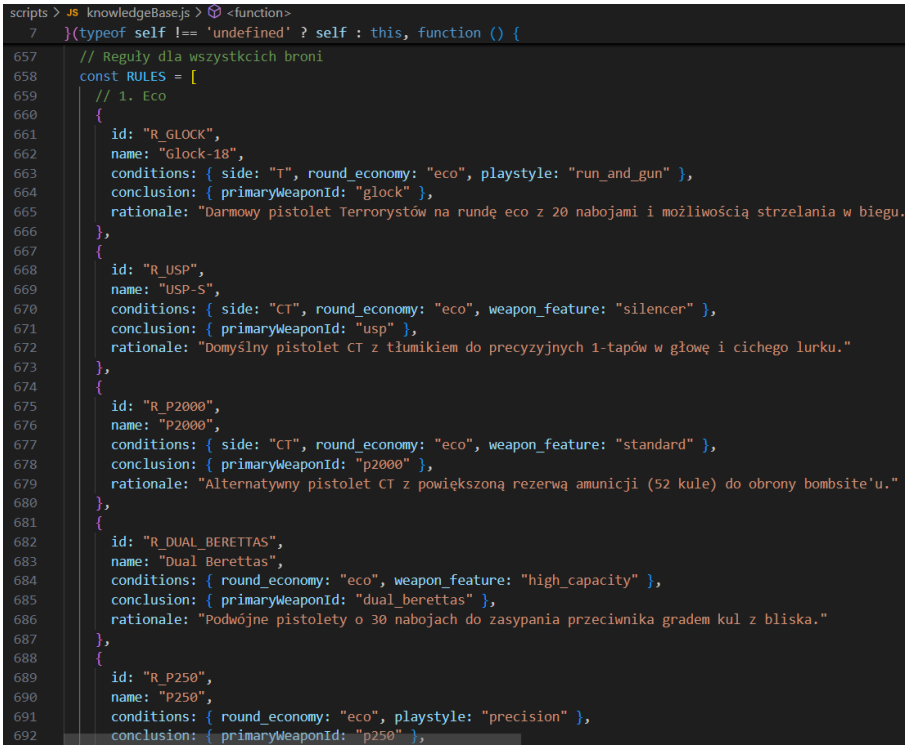
4.3. Plik interferenceEngine.js

scripts > JS inferenceEngine.js > <function> > SimpleInferenceEngine > run

```
7   }(typeof self !== 'undefined' ? self : this, function (KB) {
18    class SimpleInferenceEngine {
24      run(facts) {
51        // 3. Weryfikacja faktów
52        let matchScore = budgetScore;
53        let hasConflict = false;
54
55        for (const [key, val] of Object.entries(rule.conditions)) {
56          if (key === "side" || key === "round_economy") continue;
57          if (facts[key] === val) {
58            matchScore += 3;
59          } else if (facts[key] !== undefined) {
60            hasConflict = true;
61          }
62        }
63
64        // 4. Kara za ewentualne sprzeczności
65        if (hasConflict) {
66          matchScore -= 5;
67        }
68
69        matchedCandidates.push({
70          rule,
71          score: matchScore
72        });
73      }
74
75      // Wybór najlepiej dopasowanej reguły
76      let selectedWeaponId = null;
77      let matchedRule = null;
78
79      if (matchedCandidates.length > 0) {
80        matchedCandidates.sort((a, b) => b.score - a.score);
81        matchedRule = matchedCandidates[0].rule;
82        selectedWeaponId = matchedRule.conclusion.primaryWeaponId;
83      } else {
84        selectedWeaponId = FALLBACK_WEAPONS[budget] ? FALLBACK_WEAPONS[budget][side] : "ak47";
```

Rysunek nr 3: “Fragment kodu pliku interferenceEngine.js”

4.4. Plik knowledgeBase.js



Rysunek nr 4: “Fragment kodu pliku knowledgeBase.js”

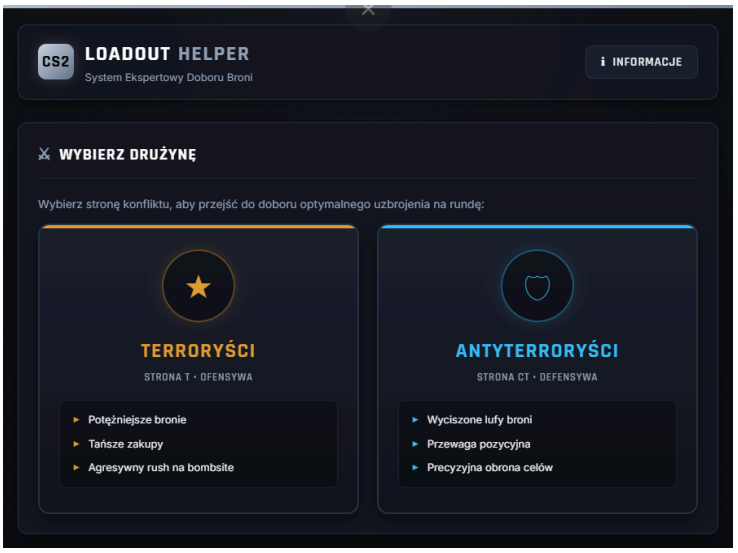
5. Interfejs użytkownika

5.1. System design i inspiracje

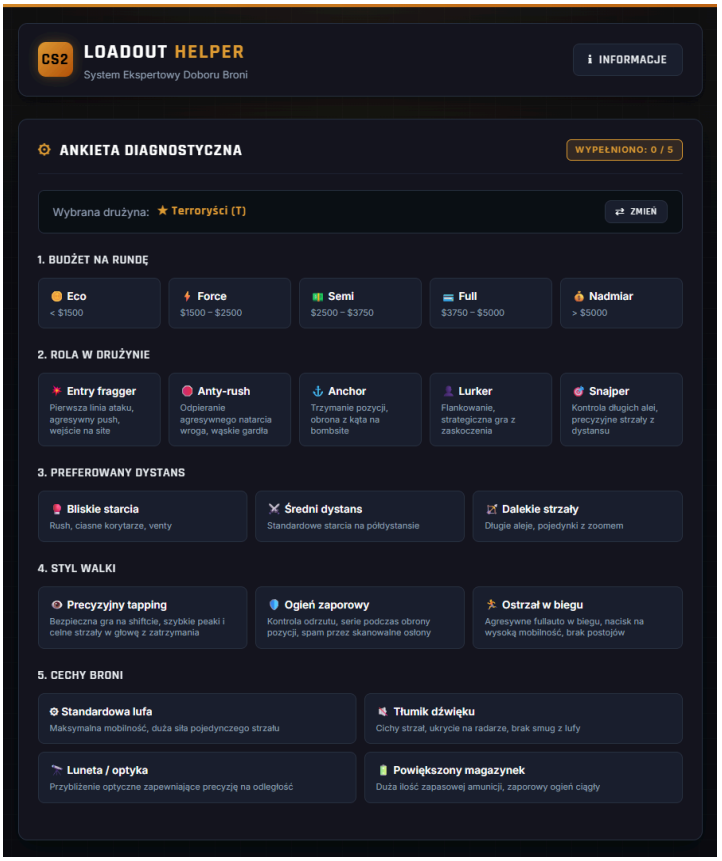
Projekt interfejsu wizualnego czerpie bezpośrednie inspiracje z estetyki gry Counter-Strike 2 oraz platformy Steam, stawiając na ciemną paletę barw (Dark Mode) i neonowe, kontrastowe akcenty. Zastosowano spójny system wzornictwa bazujący na czytelnej typografii bezszeryfowej, subtelnych cieniach oraz zaokrągleniach elementów kart ekwipunku, co nadaje całości nowoczesny i immersyjny charakter.

Inspirując się dalej HUD-em gry CS2 zaimplementowano również interaktywne, zależne od drużyny elementy dekoracyjne zmieniające kolor w zależności, czy wybrano frakcję antyterrorystów, czy terrorystów. Kluczowym założeniem było stworzenie środowiska wizualnie znajomego graczom, co obniża próg wejścia i sprawia, że obsługa aplikacji jest intuicyjna.

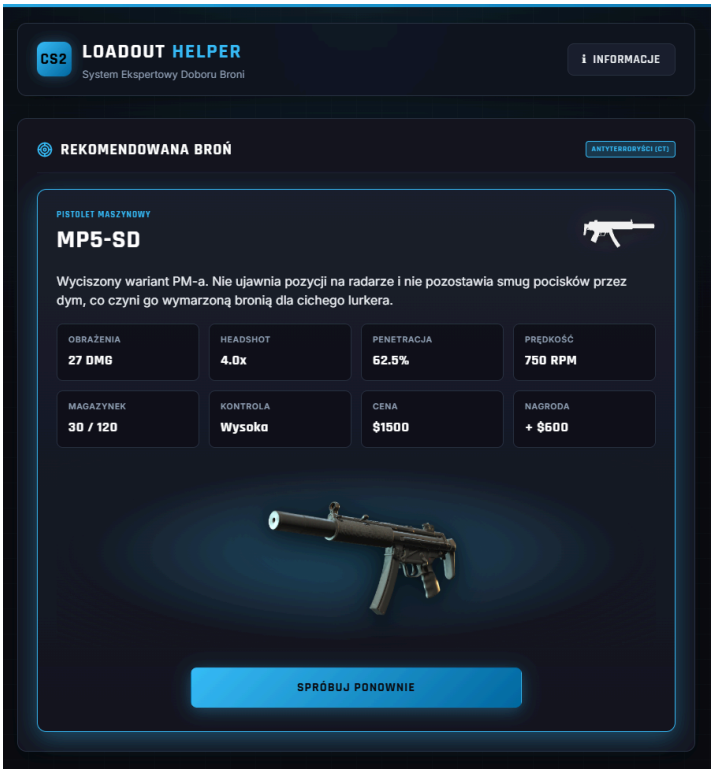
5.2. Graficzny interfejs użytkownika



Rysunek nr 5: “Wygląd interfejsu GUI panelu wyboru drużyny”



Rysunek nr 6: “Wygląd interfejsu GUI panelu ankiety diagnostycznej”



Rysunek nr 7: “Wygląd interfejsu GUI panelu rekomendowanej broni”

6. Podsumowanie

6.1. Wnioski

Zrealizowany projekt dowiódł wysokiej skuteczności zastosowania regułowych systemów ekspertowych w dziedzinie modelowania decyzji taktycznych w grach e-sportowych. Implementacja bazy wiedzy opartej na strukturze reguł typu IF–THEN w połączeniu z dedykowanym silnikiem wnioskowania w przód (Forward Chaining) pozwoliła na wierne odwzorowanie procesu decyzyjnego doświadczonego gracza Counter-Strike 2.

Zastosowany mechanizm ewaluacji reguł, uwzględniający priorytetyzację budżetową, zbieżność cech oraz penalizację sprzeczności, zapewnia stabilne i optymalne konkluzje nawet w przypadku nietypowych kombinacji parametrów wejściowych. Istotnym osiągnięciem projektu jest stworzenie w pełni autonomicznego, bezserwerowego rozwiązania działającego po stronie klienta (Client-Side), co gwarantuje natychmiastowy czas odpowiedzi silnika wnioskowania przy zachowaniu zerowej liczby zewnętrznych zależności.

Projekt potwierdził, że klasyczne metody sztucznej inteligencji doskonale sprawdzają się w deterministycznych problemach optymalizacyjnych o ściśle zdefiniowanych regułach dziedzinowych.

6.2. Możliwości rozwoju

Głównym kierunkiem dalszego rozwoju systemu ekspertowego jest pogłębienie struktury bazy wiedzy o wnioskowanie wieloetapowe. Pozwoliłoby to na generowanie kompletnego ekwipunku taktycznego (full loadout), uwzględniającego dobór pistoletu pomocniczego, stopnia opancerzenia, zestawu do rozbrajania oraz dedykowanego zestawu granatów (utility) dopasowanego do konkretnej mapy i zaplanowanej taktyki zespołowej.

W warstwie algorytmicznej perspektywicznym krokiem byłoby zastąpienie sztywnych progów ekonomicznych logiką rozmytą (Fuzzy Logic) lub wprowadzenie współczynników pewności, co umożliwiłoby lepszą obsługę niepewności w zmiennych warunkach meczowych. Wartościowym rozszerzeniem byłaby również integracja z publicznym API platformy Steam lub analizatorem plików demonstracyjnych meczów, co pozwoliłoby na automatyczne zasilanie bazy faktów w czasie rzeczywistym oraz walidację i strojenie wag reguł decyzyjnych na podstawie danych z rozgrywek profesjonalnych.

7. Bibliografia

7.1. Tabela diagramów

Numer diagramu	Strona	Nazwa diagramu
1	8	Diagram działania systemu ekspertowego

7.2. Tabela rysunków

Numer rysunku	Strona	Nazwa rysunku
1	9	Struktura aplikacji
2	10	Fragment kodu pliku app.js
3	10	Fragment kodu pliku interferenceEngine.js
4	11	Fragment kodu pliku knowledgeBase.js
5	12	Wygląd interfejsu GUI panelu wyboru drużyny
6	13	Wygląd interfejsu GUI panelu ankiety diagnostycznej
7	13	Wygląd interfejsu GUI panelu rekomendowanej broni